

WEEK 7 & 8

Name: Ramagiri Rishik

Sec: OS S-6

7. Write a program that prints the addresses of code, global, static, heap, and stack variables.
Analyze the Linux process address space layout and explain each memory segment.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;

int main()
{
    int stack=10;
    int *heap=(int*)malloc(sizeof(int));

    printf("Code Segment _ : %p\n",main);
    printf("Global Segment : %p\n",&global);
    printf("Static Segment : %p\n",&s);
    printf("Heap Segment _ : %p\n",heap);
    printf("Stack Segment _ : %p\n",&stack);
    free(heap);
    return 0;
}
```

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

PS
cat /proc/PID/maps

OUTPUT

```
adminrishik@LAPTOP-BSDK1KRC:~$ mkdir os_memory
mkdir: os_memory: File exists
1 adminrishik@LAPTOP-BSDK1KRC:~$ cd os_memory
adminrishik@LAPTOP-BSDK1KRC:~/os_memory$ pwd
/home/adminrishik/os_memory
adminrishik@LAPTOP-BSDK1KRC:~/os_memory$ nano q7.c
adminrishik@LAPTOP-BSDK1KRC:~/os_memory$ gcc q7.c -o q7
adminrishik@LAPTOP-BSDK1KRC:~/os_memory$ ls
os_memory  q7  q7.c
adminrishik@LAPTOP-BSDK1KRC:~/os_memory$ ./q7
PID: 1385
Code Segment   : 0x619a0f11d209
Global Segment : 0x619a0f120010
Static Segment : 0x619a0f120014
Heap Segment   : 0x619a2038a010
Stack Segment  : 0x7ffc7057b27c

Press Enter to exit...
```

```

1 adminrishik@LAPTOP-BSDK1KRC:~$ cat /proc/1385/maps
619a0f11c000-619a0f11d000 r--p 00000000 08:30 49144 /home/adminrishik/os_memory/q7
619a0f11d000-619a0f11e000 r-xp 00001000 08:30 49144 /home/adminrishik/os_memory/q7
619a0f11e000-619a0f11f000 r--p 00002000 08:30 49144 /home/adminrishik/os_memory/q7
619a0f11f000-619a0f120000 r--p 00002000 08:30 49144 /home/adminrishik/os_memory/q7
619a0f120000-619a0f121000 rw-p 00003000 08:30 49144 /home/adminrishik/os_memory/q7
619a2038a000-619a203ab000 rw-p 00000000 00:00 0 [heap]
7409e9000000-7409e9028000 r--p 00000000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7409e9028000-7409e91c0000 r-xp 00028000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7409e91c0000-7409e920e000 r--p 001c0000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7409e920e000-7409e9212000 r--p 0020d000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7409e9212000-7409e9214000 rw-p 00211000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6
7409e9214000-7409e9221000 rw-p 00000000 00:00 0
7409e9309000-7409e930c000 rw-p 00000000 00:00 0
7409e9312000-7409e9314000 rw-p 00000000 00:00 0
7409e9314000-7409e9318000 r--p 00000000 00:00 0 [vvar]
7409e9318000-7409e931a000 r--p 00000000 00:00 0 [vvar_vclock]
7409e931a000-7409e931c000 r-xp 00000000 00:00 0 [vdso]
7409e931c000-7409e931d000 r--p 00000000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7409e931d000-7409e934c000 r-xp 00001000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7409e934c000-7409e9357000 r--p 00030000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7409e9357000-7409e9359000 r--p 0003b000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7409e9359000-7409e935a000 rw-p 0003d000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7409e935a000-7409e935b000 rw-p 00000000 00:00 0
7ffc7055c000-7ffc7057d000 rw-p 00000000 00:00 0 [stack]
adminrishik@LAPTOP-BSDK1KRC:~$ cat /proc/1385/maps | grep heap
619a2038a000-619a203ab000 rw-p 00000000 00:00 0 [heap]
adminrishik@LAPTOP-BSDK1KRC:~$ cat /proc/1385/maps | grep stack

```

8. Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

Program:

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a,*b;

    a=(int*)malloc(5*sizeof(int));

    b=(int*)calloc(5,sizeof(int));

    a=(int*)realloc(a,10*sizeof(int));

    free(a);
    free(b);

    return 0;
}

```

OUTPUT

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano q8.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc q8.c -o q8
adminrishik@LAPTOP-BSDK1KRC:~$ ./q8
Allocating memory using malloc...
Allocating memory using calloc...
Reallocating memory for a...
Freeing memory...
All memory freed successfully.
adminrishik@LAPTOP-BSDK1KRC:~$ █

```

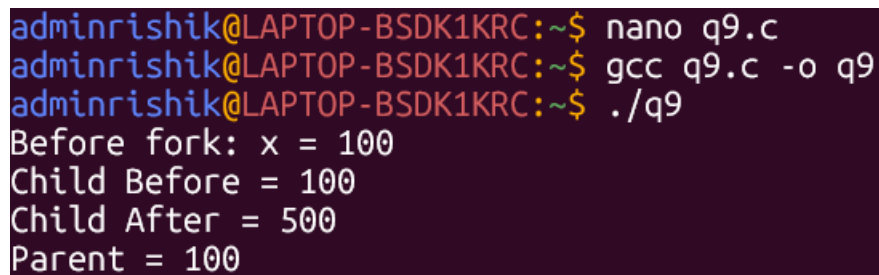
Run `valgrind` and check the result as: All heap blocks were freed -- no leaks are possible.

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int x=100;
    if(fork()==0)
    {
        printf("Child Before=%d\n",x);
        x=500;
        printf("Child After=%d\n",x);
    }
    else
    {
        sleep(2);
        printf("Parent=%d\n",x);
    }

    return 0;
}
```

OUTPUT



```
adminrishik@LAPTOP-BSDK1KRC:~$ nano q9.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc q9.c -o q9
adminrishik@LAPTOP-BSDK1KRC:~$ ./q9
Before fork: x = 100
Child Before = 100
Child After = 500
Parent = 100
```